

Contents

Documentação Consolidada — BioAcervo	3
> APOO (Wazlawick) + Arquitetura C4 + Diagramas Mermaid.	3
Guia de Leitura da Documentação APOO	3
Objetivo	3
Ordem recomendada de leitura	3
Escopo da documentação	4
Fontes de verdade usadas nesta etapa	4
Observação importante	4
Sumário Executivo	4
1. Apresentação	4
2. Problema que o sistema busca resolver	4
3. Objetivos do sistema	4
4. Público-alvo	5
5. Capacidades de negócio suportadas	5
6. Benefícios esperados	5
7. Visão geral da solução	5
8. Premissas e restrições iniciais	5
9. Limites desta versão do sumário	5
Visão Geral e Escopo	5
1. Propósito deste documento	5
2. Visão geral do sistema	6
3. Fronteiras do sistema	6
4. Escopo da documentação	6
5. Fontes de verdade	6
Atores e Glossário	6
1. Atores do sistema	6
2. Glossário	7
Requisitos Funcionais	7
1. Convenções	7
2. Acesso e usuários	7
3. Taxonomia e Localidades	7
4. Espécimes	8
5. Imagens e Etiquetas	8
6. Empréstimos	8
7. Exportação	8
8. Rastreabilidade inicial com os casos de uso	8
Requisitos Não-Funcionais	9
1. Convenções	9
2. Tecnologia e Plataforma	9
3. Persistência e Integridade	9
4. Segurança	9
5. Interoperabilidade	9
6. Operação	9
7. Observações	9
Catálogo de Casos de Uso	9
1. Convenções	9
2. Acesso e usuários	9
UC-01 Cadastrar usuário no sistema	9

UC-02 Autenticar usuário	10
UC-03 Atualizar próprio perfil	10
3. Taxonomia e Localidades	10
UC-04 Cadastrar Taxonomia	10
UC-05 Cadastrar Localidade Geográfica	10
4. Espécimes	10
UC-06 Cadastrar Espécime	10
UC-07 Buscar Espécimes por Bounding Box	10
UC-08 Associa Imagem a Espécime	10
5. Empréstimos	11
UC-09 Emprestar Espécime	11
UC-10 Retornar Espécime Emprestado	11
6. Exportação	11
UC-11 Exportar Darwin Core Archive	11
7. Casos prioritários para expansão	11
Casos de Uso Expandidos	11
1. Convenções	11
2. UC-06 — Cadastrar Espécime (expandido)	11
3. UC-07 — Buscar por Bounding Box (expandido)	12
4. UC-09 — Emprestar Espécime (expandido)	12
5. UC-10 — Retornar Espécime (expandido)	12
6. UC-11 — Exportar DwC-A (expandido)	12
Regras de Negócio Consolidadas	12
1. Objetivo	12
2. Convenções	12
3. Usuários e acesso	12
4. Taxonomia e Localidades	13
5. Espécimes	13
6. Empréstimos	13
7. Exportação	13
8. Observações para manutenção	13
Máquinas de Estado	13
1. Objetivo	13
2. Usuário (PerfilUsuario + ativo)	13
3. Espécime (StatusEspecime)	13
4. Empréstimo (ativo)	14
5. Observações	14
Análise e Modelo Conceitual	14
1. Objetivo	14
2. Fonte de verdade	14
3. Entidades centrais	14
Usuario	14
Taxonomia	14
LocalidadeGeografica	14
Especime	15
ImagemEspecime	15
Emprestimo	15
4. Enumerações	15
5. Observações	15
Projeto Arquitetural	16
1. Visão geral	16

2. Stack	16
3. Camadas (C4 — Nível de Contêiner)	17
4. Camadas internas (Backend)	17
5. Fluxo de requisição	17
7. Decisões arquiteturais	17
5. Diagramas	18
Padrões de Projeto e Diretrizes de Extensão	19
1. Objetivo	19
2. Padrões de backend	19
3. Modelo de dados	19
4. Autenticação e autorização	19
5. Exportação	19
6. Diretrizes de extensão	19
Rastreabilidade e Priorização	20
1. Objetivo	20
2. Matriz RF → UC → RN	20
3. Priorização inicial	20
4. Observações	20

Documentação Consolidada — BioAcervo

> APOO (Wazlawick) + Arquitetura C4 + Diagramas Mermaid.

Guia de Leitura da Documentação APOO

Objetivo

Esta pasta concentra a documentação formal do sistema **BioAcervo** (Herbário Digital) no formato de Análise e Projeto Orientados a Objetos (Wazlawick), usado no IFNMG Campus Januária.

Ela foi estruturada para atender dois objetivos simultâneos: - apoiar leitura acadêmica e de projeto - facilitar onboarding e manutenção por novos desenvolvedores

Ordem recomendada de leitura

1. 01-sumario-executivo.md
2. 02-visao-geral-e-escopo.md
3. 03-atores-e-glossario.md
4. 04-requisitos-funcionais.md
5. 05-requisitos-nao-funcionais.md
6. 06-casos-de-uso-catalogo.md
7. 07-casos-de-uso-expandidos.md
8. 08-regras-de-negocio.md
9. 09-maquinas-de-estado.md
10. 10-analise-e-modelo-conceitual.md
11. 11-projeto-arquitetural.md
12. 12-padroes-de-projeto-e-diretrizes-de-extensao.md
13. 13-rastreabilidade-e-priorizacao.md

Depois da leitura funcional, seguir para os artefatos técnicos:

- Architecture/00_Arquitetura.md (visão em camadas C4)
- Finals/AP00_Herbario_Digital.pdf (documento consolidado)

Escopo da documentação

Esta documentação não descreve cada função do código-fonte nem cada detalhe de interface.

Ela prioriza: - processos suportados pelo sistema - atores e responsabilidades - requisitos - casos de uso - estados e regras relevantes - arquitetura e manutenção futura

Fontes de verdade usadas nesta etapa

- backend/app/models/models.py
- backend/app/schemas/schemas.py
- backend/app/services/*.py
- backend/app/api/v1/endpoints/*.py
- backend/app/core/security.py, backend/app/core/config.py
- README.md, docker-compose.yml

Observação importante

O backend usa SQLAlchemy ORM assíncrono (asyncpg) e FastAPI. O modelo de dados em `models.py` é a fonte de verdade para atributos e restrições; os endpoints em `api/v1/endpoints/` são a fonte para casos de uso e regras de acesso.

Sumário Executivo

1. Apresentação

O **BioAcervo** é uma API RESTful de Gestão de Acervo Biológico (herbário digital), construída com **FastAPI** e **PostgreSQL** (acesso via SQLAlchemy ORM assíncrono). Sua proposta é registrar, catalogar, identificar taxonomicamente, georreferenciar e disponibilizar espécimes biológicos, além de controlar seu empréstimo entre instituições e exportar os dados no padrão **Darwin Core Archive (DwC-A)** para portais de biodiversidade (GBIF, iDigBio, SpeciesLink).

O sistema combina cadastro de espécimes, classificação taxonômica, geolocalização, imagens, controle de acesso por perfil e exportação padronizada. Seu objetivo principal é dar rastreabilidade, padronização (DwC) e continuidade ao acervo do herbário.

2. Problema que o sistema busca resolver

O contexto identificado no repositório indica a necessidade de resolver, ao mesmo tempo, problemas de organização e de padronização: - dispersão do acervo em planilhas e registros manuais de coleta - falta de padronização taxonômica e geográfica entre coletores - dificuldade de rastrear procedência, status e localização física de cada espécime - ausência de controle de empréstimo entre instituições - impedimento de integração com portais de biodiversidade por falta de exportação DwC-A

3. Objetivos do sistema

Os objetivos centrais do BioAcervo são: - centralizar o acervo biológico em uma única plataforma - catalogar espécimes com código de catálogo e código de barras únicos - identificar taxonomicamente cada espécime (taxonomia + sinônimos) - georreferenciar a coleta (latitude, longitude, altitude, datum, precisão) - registrar imagens do espécime (JPEG/PNG/TIFF/WebP até 20 MB) - controlar empréstimo de espécimes com marcão de status - exportar os dados em DwC-A (occurrence.csv + meta.xml + eml.xml)

De forma mais específica, o sistema busca: - apoiar a curadoria do acervo (cadastro, edição, baixa) - oferecer busca avançada por critérios taxonômicos, geográficos, temporais e bounding box - gerar etiqueta PDF com Code128 - garantir conformidade com Darwin Core para portais externos

4. Público-alvo

Os perfis de usuário identificados no sistema (enum `PerfilUsuario`) são: - **Curador**: cadastra e edita espécimes, taxonomias, localidades e empréstimos - **Administrador**: acesso total, inclusive gestão de usuários e remoção definitiva de registros - **Leitor**: acesso somente leitura (consulta do acervo)

Instituições de pesquisa são destinatárias de empréstimos de espécimes.

5. Capacidades de negócio suportadas

O sistema suporta, em nível executivo, os seguintes grupos de capacidade: - cadastro, autenticação e gestão de usuários por perfil - CRUD completo de Espécimes, Taxonomias, Localidades Geográficas e Imagens - controle de Empréstimos (saída, retorno, status) - georreferenciamento e busca por caixa geográfica (bounding box) - upload e associação de imagens ao espécime - geração de etiqueta PDF com código de barras - exportação DwC-A (ZIP com occurrence.csv, meta.xml, eml.xml)

Essas capacidades atendem a uma proposta de centralização e padronização do acervo, com foco em conformidade Darwin Core.

6. Benefícios esperados

Os principais benefícios esperados com a utilização do sistema são: - maior rastreabilidade sobre procedência e status de cada espécime - padronização taxonômica e geográfica (DwC) - histórico consistente de empréstimos - integração com portais de biodiversidade (GBIF, iDigBio, SpeciesLink) - melhor onboarding de novos curadores

7. Visão geral da solução

Do ponto de vista tecnológico, o sistema foi implementado como uma API web (FastAPI) com persistência relacional (PostgreSQL via SQLAlchemy assíncrono). A solução adota: - interface web/API para uso cotidiano (endpoints REST) - backend modular por domínio (services) + camada de API (endpoints) - banco relacional PostgreSQL com esquema em `models.py` - controle de autenticação e autorização via JWT Bearer Token

8. Premissas e restrições iniciais

No contexto atual do projeto, foram informadas as seguintes premissas: - o sistema opera com perfis de acesso (leitor, curador, administrador) definidos em `PerfilUsuario` - a exportação DwC-A é a via de integração com portais externos - imagens são armazenadas em bucket local (volume Docker)

Essas premissas devem ser levadas em conta ao interpretar os requisitos não funcionais e as decisões arquiteturais registradas nos demais documentos.

9. Limites desta versão do sumário

Este sumário executivo foi produzido a partir da base de código e da documentação técnica existente. Portanto, ele descreve com segurança: - o que o sistema faz - quem aparenta usar o sistema - quais processos são suportados - quais preocupações arquiteturais e operacionais estão presentes

Permanecem como pontos que podem ser refinados em versões futuras: - detalhamento de requisitos de privacidade e conformidade ligados à LGPD - endurecimento de segurança para ambiente além da rede local - definição de indicadores formais de sucesso e adoção - eventual mapeamento de processos externos ou integrações futuras

Visão Geral e Escopo

1. Propósito deste documento

Este documento descreve a visão geral do **BioAcervo** e o escopo da documentação APOO. Ele serve como ponte entre o Sumário Executivo e os requisitos detalhados.

2. Visão geral do sistema

O BioAcervo é uma API de acervo biológico que centraliza: - **Catálogo** de espécimes (código de catálogo + código de barras únicos) - **Identificação taxonômica** (reino → espécie, com sinônimos) - **Georreferenciamento** da coleta (lat/long, altitude, datum, precisão) - **Imagens** do espécime (JPEG/PNG/TIFF/WebP até 20 MB) - **Empréstimo** de espécimes entre instituições - **Exportação Darwin Core Archive (DwC-A)** para GBIF/iDigBio/SpeciesLink

3. Fronteiras do sistema

Dentro do escopo	Fora do escopo (versão atual)
CRUD de Espécimes, Taxonomias, Localidades	Interface web (frontend não documentado nesta APOO)
Autenticação JWT + RBAC por perfil	Federção de identidade externa
Busca avançada (taxonômica, geográfica, bounding box)	Análise filogenética
Upload de imagens + etiqueta PDF Code128	Curadoria de dados de terceiros
Empréstimo com mudança de status	Workflow de publicação científica
Export DwC-A (ZIP)	Integração direta com portais (push)

4. Escopo da documentação

Esta APOO cobre: - Requisitos funcionais (RF-01 a RF-20) - Requisitos não-funcionais (RNF-01 a RNF-06) - Casos de uso (UC-01 a UC-14) - Regras de negócio (RN-01 a RN-12) - Máquinas de estado (Usuário, Espécime, Empréstimo) - Modelo conceitual (entidades e associações) - Projeto arquitetural (camadas C4) - Rastreabilidade RF → UC → RN

5. Fontes de verdade

- backend/app/models/models.py — entidades e enums
- backend/app/schemas/schemas.py — DTOs Pydantic v2
- backend/app/services/*.py — lógica de negócio
- backend/app/api/v1/endpoints/*.py — exposição REST
- backend/app/core/security.py — JWT/RBAC
- README.md, docker-compose.yml

Atores e Glossário

1. Atores do sistema

Ator	Perfil (PerfilUsuario)	Responsabilidade
Curador	curador	Cadastra e edita espécimes, taxonomias, localidades e empréstimos
Administrador	administrador	Acesso total, gestão de usuários e remoção definitiva de registros
Leitor	leitor	Apenas consulta o acervo (somente leitura)
Instituição de pesquisa	externa	Destinatária de empréstimos de espécimes

2. Glossário

Termo	Definição
Espécime	Indivíduo biológico coletado, catalogado e georreferenciado
Taxonomia	Classificação científica (reino → espécie) de um espécime
Localidade Geográfica	Ponto de coleta (país, estado, município, lat/long, altitude, datum)
Código de Catálogo	Identificador único do espécime no acervo (ex.: A11.1)
Código de Barras	Identificador SPEC-<uuid> para etiquetas físicas
DwC Record ID	URN urn:uuid:<uuid> no padrão Darwin Core
Darwin Core Archive (DwC-A)	Pacote ZIP com <code>occurrence.csv</code> , <code>meta.xml</code> , <code>eml.xml</code> para GBIF/iDigBio/SpeciesLink
Empréstimo	Saída temporária de espécime para instituição externa
Status do Espécime	ativo emprestado (enum <code>StatusEspecime</code>)
Tipo de Coleta	campo cultivo aves mamífero (enum <code>TipoColeta</code>)
Bucket Local	Volume Docker para armazenamento de imagens
JWT Bearer Token	Mecanismo de autenticação via <code>Authorization: Bearer <token></code>

Requisitos Funcionais

1. Convenções

- Identificador de requisito funcional: **RF-XX**
- Sempre que possível, cada requisito deve se ligar a um ou mais casos de uso
- Requisitos derivados diretamente do comportamento observado no código podem ser refinados na fase de casos de uso expandidos

2. Acesso e usuários

- RF-01 O sistema deve permitir o cadastro de novos usuários com nome, email e senha (`usuarios_emprestimos.py:39`)
- RF-02 O sistema deve registrar novos usuários com perfil padrão leitor (`models.py:PerfilUsuario, usuario_service.create`)
- RF-03 O sistema deve permitir autenticação por JWT Bearer Token (`core/security.py:get_current_user`)
- RF-04 O sistema deve impedir acesso autenticado a usuários sem perfil autorizado (`require_roles`)
- RF-05 O sistema deve permitir a criação de usuários apenas por administrador (`usuarios_emprestimos.py:43`)
- RF-06 O sistema deve permitir atualização de próprio perfil pelo usuário (`usuarios_emprestimos.py:65, UsuarioSelfUpdate`)
- RF-07 O sistema deve impedir que não-admin altere perfil ou status de usuário (`usuarios_emprestimos.py:74`)

3. Taxonomia e Localidades

- RF-08 O sistema deve permitir CRUD de Taxonomias por administrador/curador (`taxonomia_localidade.py:53, require_roles("administrador", "curador")`)
- RF-09 O sistema deve garantir `nome_cientifico` único e não nulo (`models.py:Taxonomia, UNIQUE`)
- RF-10 O sistema deve permitir CRUD de Localidades Geográficas por administrador/curador (`taxonomia_localidade.py:129`)
- RF-11 O sistema deve validar `latitude/longitude` como `FLOAT` e `datum_geodesico` padrão WGS84 (`models.py:LocalidadeGeografica`)

4. Espécimes

- RF-12 O sistema deve permitir cadastro de Espécime com geração automática de `codigo_barras` (SPEC-<uuid>) e `dwc_record_id` (urn:uuid:<uuid>) (`especime_service.py:46-54`)
- RF-13 O sistema deve exigir `taxonomia_id` (FK NOT NULL) em todo Espécime (`models.py:Especime`)
- RF-14 O sistema deve permitir busca avançada por nome científico, família, gênero, estado, município, bioma (`especime_service.buscar, ilike`)
- RF-15 O sistema deve permitir busca por caixa geográfica (bounding box: lat/lon min/max) (`especime_service.buscar:125-133`)
- RF-16 O sistema deve permitir busca por coletor, status e intervalo de data de coleta (`especime_service.buscar:135-149`)
- RF-17 O sistema deve permitir edição de Espécime apenas por `administrador/curador` (`especimes.py + require_roles`)
- RF-18 O sistema deve impedir exclusão de Espécime com imagens associadas (ON DELETE CASCADE em `ImagemEspecime`)

5. Imagens e Etiquetas

- RF-19 O sistema deve permitir upload de imagens (JPEG/PNG/TIFF/WebP até 20 MB) vinculadas ao Espécime (`imagem_service.py`)
- RF-20 O sistema deve gerar etiqueta PDF com Code128 a partir do `codigo_barras` (`export_service.EtiquetaService`)

6. Empréstimos

- RF-21 O sistema deve permitir criar Empréstimo de Espécime por `administrador/curador` (`usuarios_emprestimos.py:`
- RF-22 O sistema deve impedir empréstimo de Espécime com `status != "ativo"` (`usuarios_emprestimos.py:134`)
- RF-23 O sistema deve marcar `status = "emprestado"` no Espécime ao criar empréstimo (`usuarios_emprestimos.py:141`)
- RF-24 O sistema deve marcar `status = "ativo"` no Espécime e `ativo = False` no Empréstimo ao retornar (`usuarios_emprestimos.py:162-166`)

7. Exportação

- RF-25 O sistema deve exportar os dados em Darwin Core Archive (ZIP: `occurrence.csv + meta.xml + eml.xml`) (`export_service.ExportService`)

8. Rastreabilidade inicial com os casos de uso

O mapeamento detalhado entre requisitos funcionais e casos de uso será consolidado na fase seguinte, no catálogo e na matriz de rastreabilidade.

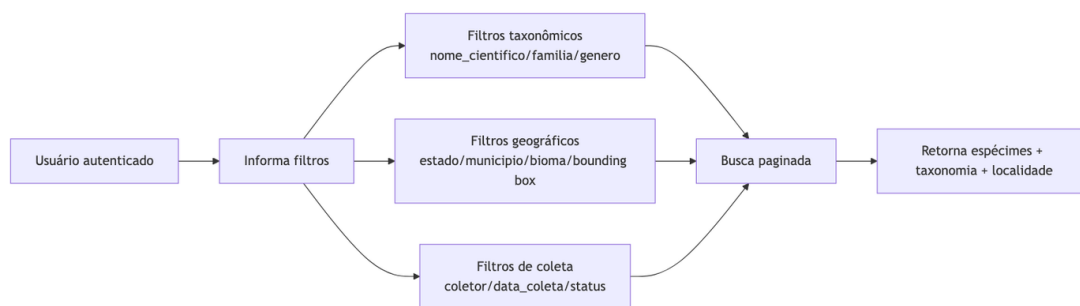


Figure 1: Fluxo de Busca Avançada

Requisitos Não-Funcionais

1. Convenções

- Identificador de requisito não-funcional: **RNF-XX**
- Requisitos derivados da estrutura do projeto (Docker, SQLAlchemy assíncrono, Pydantic v2)

2. Tecnologia e Plataforma

- RNF-01 O backend deve usar FastAPI + SQLAlchemy ORM assíncrono (asyncpg) (`core/session.py`)
- RNF-02 O banco de dados deve ser PostgreSQL 16 (container `bioacervo-database`) (`docker-compose.yml`)
- RNF-03 A validação de entrada deve usar Pydantic v2 (DTOs em `schemas.py`)

3. Persistência e Integridade

- RNF-04 O sistema deve garantir unicidade de `email` (Usuário), `nome_cientifico` (Taxonomia, com índice trigram), `codigo_catalogo` e `codigo_barras` (Espécime), `dwc_record_id` (`models.py`)
- RNF-05 Relações de exclusão em cascata devem ser aplicadas (`ImagemEspecime` → `Especime` ON DELETE CASCADE) (`models.py:ImagemEspecime`)

4. Segurança

- RNF-06 Senhas devem ser armazenadas com hash (bcrypt) e autenticação via JWT (`core/security.py`)
- RNF-07 Controle de acesso por perfil (`PerfilUsuario`: leitor/curador/administrador) via `require_roles`

5. Interoperabilidade

- RNF-08 A exportação deve seguir o padrão Darwin Core Archive (`occurrence.csv` + `meta.xml` + `eml.xml`) para GBIF/iDigBio/SpeciesLink (`export_service.py`)

6. Operação

- RNF-09 Imagens devem ser armazenadas em bucket local (volume Docker) com limite de 20 MB por arquivo (`imagem_service.py`)
- RNF-10 O sistema deve expor saúde da API em `/health` (`main.py`)

7. Observações

Estes requisitos refletem o estado atual do `docker-compose.yml` e de `models.py`. Ajustes futuros (LGPD, hardening de rede) devem ser registrados como trabalho futuro.

Catálogo de Casos de Uso

1. Convenções

- Identificador de caso de uso: **UC-XX**
- Este catálogo resume os casos de uso do sistema sem expandir todos os fluxos
- Apenas os casos mais relevantes serão detalhados em versão expandida

2. Acesso e usuários

UC-01 Cadastrar usuário no sistema

- Objetivo: permitir que um admin registre um novo usuário
- Atores primários: administrador
- Gatilho: o admin acessa a tela de criação de usuário

- Pré-condições: permissão `administrador`
- Pós-condições: usuário criado (`models.py:UsuarioService.create`)

UC-02 Autenticar usuário

- Objetivo: permitir acesso autenticado via JWT
- Atores primários: usuário cadastrado
- Gatilho: o usuário informa credenciais na tela de login
- Pré-condições: conta existente
- Pós-condições: token JWT emitido (`core/security.py`)

UC-03 Atualizar próprio perfil

- Objetivo: manter dados de perfil atualizados
- Atores primários: usuário autenticado
- Gatilho: o usuário acessa a área de perfil
- Pré-condições: sessão autenticada
- Pós-condições: perfil persistido (`UsuarioSelfUpdate`)

3. Taxonomia e Localidades

UC-04 Cadastrar Taxonomia

- Objetivo: registrar classificação científica
- Atores: curador, administrador
- Pré: `require_roles("administrador", "curador")`
- Pós: Taxonomia criada (`taxonomia_localidade.py:53`)

UC-05 Cadastrar Localidade Geográfica

- Objetivo: registrar ponto de coleta
- Atores: curador, administrador
- Pós: LocalidadeGeografica criada (`taxonomia_localidade.py:129`)

4. Espécimes

UC-06 Cadastrar Espécime

- Objetivo: registrar espécime com código de barras e DwC ID automáticos
- Atores: curador, administrador
- Gatilho: formulário de criação de espécime
- Pós: Espécime criado com `codigo_barras=SPEC-<uuid>`, `dwc_record_id=urn:uuid:<uuid>` (`especime_service.py:46-54`)

UC-07 Buscar Espécimes por Bounding Box

- Objetivo: filtrar espécimes por caixa geográfica
- Atores: usuário autenticado
- Gatilho: informa lat/lon min/max
- Pós: lista paginada de espécimes (`especime_service.buscar:125-133`)

UC-08 Associa Imagem a Espécime

- Objetivo: vincular imagem (até 20 MB) ao espécime
- Atores: curador, administrador
- Pós: ImagemEspecime criada (`imagem_service.py`)

5. Empréstimos

UC-09 Empréstimo Espécime

- Objetivo: registrar saída de espécime para instituição
- Atores: curador, administrador
- Pré: espécime com `status == "ativo"`
- Pós: Espécime `status="emprestado"` (usuarios_emprestimos.py:134-141)

UC-10 Retornar Espécime Empréstado

- Objetivo: registrar retorno e reativar espécime
- Atores: curador, administrador
- Gatilho: informa `data_retorno`
- Pós: Espécime `status="ativo"`, Empréstimo `ativo=False` (usuarios_emprestimos.py:162-166)

6. Exportação

UC-11 Exportar Darwin Core Archive

- Objetivo: gerar ZIP DwC-A (occurrence.csv + meta.xml + eml.xml)
- Atores: curador, administrador
- Pós: arquivo DwC-A disponibilizado (export_service.py)

7. Casos prioritários para expansão

- UC-01, UC-02, UC-04, UC-05, UC-06, UC-07, UC-09, UC-10, UC-11

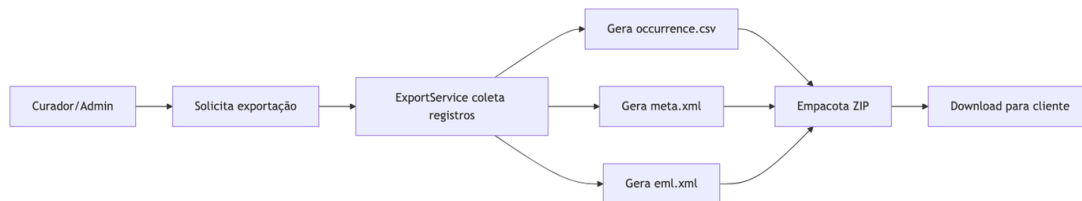


Figure 2: Fluxo de Exportação DwC-A

Casos de Uso Expandidos

1. Convenções

- Cada caso abaixo detalha fluxo, entradas e exceções conforme `endpoints/` e `services/`
- Identificador mantém o padrão UC-XX do catálogo

2. UC-06 — Cadastrar Espécime (expandido)

Atores: curador, administrador **Pré-condição:** sessão autenticada com perfil autorizado; `taxonomia_id` válido
Fluxo principal: 1. O ator aciona criação de espécime (`especimes.py` POST `/especimes`) 2. O endpoint valida JWT e `requires_roles("administrador", "curador")` 3. `EspecimeService.create` gera `codigo_barras = SPEC-<uuid>[:12]` e `dwc_record_id = urn:uuid:<uuid>` (`especime_service.py`:46-54) 4. O serviço persiste `data_entrada_colecao = utcnow()` e `cadastrado_por_id = usuario_id` 5. Retorna espécime com relacionamentos carregados

Exceções: `taxonomia_id` inválido → 404; `localidade_id` opcional

3. UC-07 — Buscar por Bounding Box (expandido)

Atores: usuário autenticado **Fluxo:** 1. GET /especimes/buscar com filtros (especime_service.buscar) 2. Condições acumulativas: nome_cientifico/familia/genero (ILIKE em Taxonomia), estado/municipio/bioma (ILIKE em LocalidadeGeografica), lat/lon min/max (bounding box), coletor, status, data_coleta_inicio/fim 3. Contagem total + paginação (page/per_page, order_by id desc) 4. Retorna PaginatedResponse

4. UC-09 — Emprestar Espécime (expandido)

Atores: curador, administrador **Pré:** espécime existe e status == "ativo" **Fluxo:** 1. POST /emprestimos (usuarios_emprestimos.py:125) 2. Valida perfil e existência do espécime (404 se ausente) 3. Se status != "ativo" → 400 "Espécime não disponível" 4. Cria Empréstimo com responsavel_id = current_user.id 5. Define especime.status = "emprestado" e flush (linha 141)

5. UC-10 — Retornar Espécime (expandido)

Atores: curador, administrador **Fluxo:** 1. PUT /emprestimos/{eid} com data_retorno (usuarios_emprestimos.py:147) 2. Se data_retorno presente: especime.status = "ativo" e ativo = False (linhas 162-166) 3. Persiste alterações

6. UC-11 — Exportar DwC-A (expandido)

Atores: curador, administrador **Fluxo:** 1. ExportService coleta espécimes + taxonomias + localidades 2. Gera occurrence.csv (padão DwC), meta.xml, eml.xml 3. Empacota em ZIP retornado ao cliente (export_service.py)



Figure 3: Fluxo de Empréstimo

Regras de Negócio Consolidadas

1. Objetivo

Este documento consolida as regras de negócio mais relevantes do BioAcervo, evitando que elas fiquem dispersas entre casos de uso, rotas e implementações backend.

As regras abaixo foram extraídas da codebase atual e devem ser tratadas como referência operacional do sistema.

2. Convenções

- Identificador de regra de negócio: RN-XX
- Quando uma regra depender fortemente de estado, a máquina correspondente deve ser consultada em 09-maquinas-de-estado.md
- Quando uma regra estiver associada a um caso de uso expandido, o identificador do caso deve ser referenciado na manutenção futura

3. Usuários e acesso

- RN-01 Novos usuários são criados com perfil padrão leitor (models.py:PerfilUsuario; usuario_service.create)
- RN-02 Apenas administrador pode criar usuários (usuarios_emprestimos.py:43)
- RN-03 Usuário só atualiza próprio perfil; admin pode editar qualquer um (usuarios_emprestimos.py:65-81)
- RN-04 Não-admin não pode alterar perfil ou ativo de usuário (usuarios_emprestimos.py:74-75)

- RN-05 Leitor possui acesso somente leitura a todos os endpoints

4. Taxonomia e Localidades

- RN-06 Taxonomia exige `nome_cientifico` único e não nulo (models.py:Taxonomia, UNIQUE)
- RN-07 CRUD de Taxonomia/Localidade exige perfil `administrador` ou `curador` (taxonomia_localidade.py:57,133)
- RN-08 Localidade usa `datum_geodesico` padrão WGS84 (models.py:LocalidadeGeografica)

5. Espécimes

- RN-09 Espécime exige `taxonomia_id` (FK NOT NULL) (models.py:Especime)
- RN-10 `codigo_catalogo` e `codigo_barras` são únicos (models.py:Especime, UNIQUE)
- RN-11 Na criação, geram-se automaticamente `codigo_barras = SPEC-<uuid>[:12]` e `dwc_record_id = urn:uuid:<uuid>` (especime_service.py:46-54)
- RN-12 `coletores_adicionais` e `referencias_bibliograficas` são JSONB (models.py:Especime)
- RN-13 Exclusão de espécime remove em cascata suas imagens (ImagemEspecime ON DELETE CASCADE)

6. Empréstimos

- RN-14 Espécime só pode ser emprestado se `status == "ativo"` (usuarios_emprestimos.py:134)
- RN-15 Ao criar empréstimo, espécime vai para `status = "emprestado"` (usuarios_emprestimos.py:141)
- RN-16 Ao retornar (data_retorno informada), espécime volta a `status = "ativo"` e `empréstimo ativo = False` (usuarios_emprestimos.py:162-166)
- RN-17 Empréstimo exige `especime_id` e `responsavel_id` (FK NOT NULL)

7. Exportação

- RN-18 A exportação DwC-A gera `occurrence.csv + meta.xml + eml.xml` conforme Darwin Core (export_service.py)

8. Observações para manutenção

- Regras de acesso devem ser validadas no backend (require_roles), não apenas na UI
- Campos JSONB (`coletores_adicionais`, `referencias_bibliograficas`, `sinonimos`) exigem parse defensivo no import
- O status do espécime é a fonte de verdade para disponibilidade de empréstimo

Máquinas de Estado

1. Objetivo

Documentar os estados relevantes das entidades que possuem ciclo de vida controlado por regras de negócio.

2. Usuário (PerfilUsuario + ativo)

Estado	Transição	Gatilho	Regra
criado	leitor (padrão)	admin cria	RN-01
ativo	ativo = True	admin aprova	require_roles("administrador")
inativo	ativo = False	admin desativa	RN-04

3. Espécime (StatusEspecime)

Estado	Significado	Transição	Regra
ativo	disponível no acervo	criação ou retorno	RN-16
emprestado	fora para instituição	criação de empréstimo	RN-15 (exige status==ativo, RN-14)
retorno	ativo novamente	data_retorno informada	RN-16

4. Empréstimo (ativo)

Estado	Significado	Transição	Regra
ativo	em andamento	POST /emprestimos	RN-15
encerrado	devolvido	PUT com data_retorno	RN-16 (ativo=False)

5. Observações

- O status do espécime é a fonte de verdade para disponibilidade de empréstimo (RN-14)
- Mudanças de status devem ser validadas no backend, não na UI

Análise e Modelo Conceitual

1. Objetivo

Registrar as entidades de domínio do BioAcervo, seus atributos mais relevantes e os relacionamentos principais, tomando como fonte de verdade `backend/app/models/models.py`.

Este documento não substitui o schema. Ele serve como apoio de leitura para manutenção e análise de impacto.

2. Fonte de verdade

- `backend/app/models/models.py`

3. Entidades centrais

Usuario

Acesso ao sistema. Perfil define permissões (leitor/curador/administrador). - Campos: `id`, `nome`, `email` (único), `senha_hash`, `perfil` (ENUM), `ativo` (bool), `criado_em`, `atualizado_em` - Relacionamentos: 1-* Espécime (`cadastrado_por_id`); 1-* Empréstimo (`responsavel_id`)

Taxonomia

Classificação científica do espécime. - Campos: `id`, `reino/filo/classe/ordem`, `familia/genero`, `epiteto_especifico`, `nome_cientifico` (único, NOT NULL), `autor_descricao`, `ano_descricao`, `nome_comum`, `sinonimos` (JSONB), `notas_taxonomicas` (TEXT) - Relacionamentos: 1-* Espécime (`taxonomia_id`, NOT NULL)

LocalidadeGeografica

Ponto de coleta georreferenciado. - Campos: `id`, `pais` (default Brasil), `estado/municipio/localidade`, `latitude/longitude` (FLOAT), `altitude_m/precisao_coord`, `datum_geodesico` (WGS84), `metodo_geolocalizacao`, `bioma` - Relacionamentos: 1-* Espécime (`localidade_id`, opcional)

Especime

Entidade central. Indivíduo biológico catalogado. - Campos: `id`, `codigo_catalogo` (único), `codigo_barras` (único SPEC-), `taxonomia_id` (FK NOT NULL), `localidade_id` (FK), `data_coleta/data_coleta_fim`, `tipo_coleta` (ENUM), `coletor_principal`, `coletores_adicionais` (JSONB), `numero_campo`, `sexo/estagio_vida/condicao`, `numero_individuos`, `descricao_morfologica/habitat/obs`, `identificado_por`, `data_identificacao`, `status` (ENUM ativo/emprestado), `localizacao_fisica`, `meio_preservacao`, `dwc_record_id` (único urn:uuid), `cadastrado_por_id` (FK), `criado_em/atualizado_em` - Relacionamentos: -1 *Taxonomia*; -1 Localidade; 1-* ImagemEspecime (CASCADE); 1-* Empréstimo

ImagemEspecime

Mídia do espécime. - Campos: `id`, `especime_id` (FK NOT NULL ON DELETE CASCADE), `nome_arquivo`, `caminho`, `url_relativa`, `tamanho_bytes/largura_px/altura_px`, `descricao`, `is_principal` (bool) - Relacionamentos: *-1 Espécime

Emprestimo

Saída temporária de espécime. - Campos: `id`, `especime_id` (FK NOT NULL), `responsavel_id` (FK NOT NULL), `instituicao_destino`, `pesquisador_responsavel`, `finalidade`, `data_saida` (NOT NULL), `data_prevista_retorno`, `data_retorno`, `observacoes`, `ativo` (bool) - Relacionamentos: -1 *Espécime*; -1 Usuário (responsavel)

4. Enumerações

- `PerfilUsuario`: leitor, curador, administrador
- `StatusEspecime`: ativo, emprestado
- `TipoColeta`: campo, cultivo, aves, mamuideo

5. Observações

- `sinonimos`, `coletores_adicionais`, `referencias_bibliograficas` são JSONB
- `codigo_barras` segue padrão SPEC-`<hex>`; `dwc_record_id` é URN Darwin Core
- Imagens seguem ON DELETE CASCADE a partir do espécime

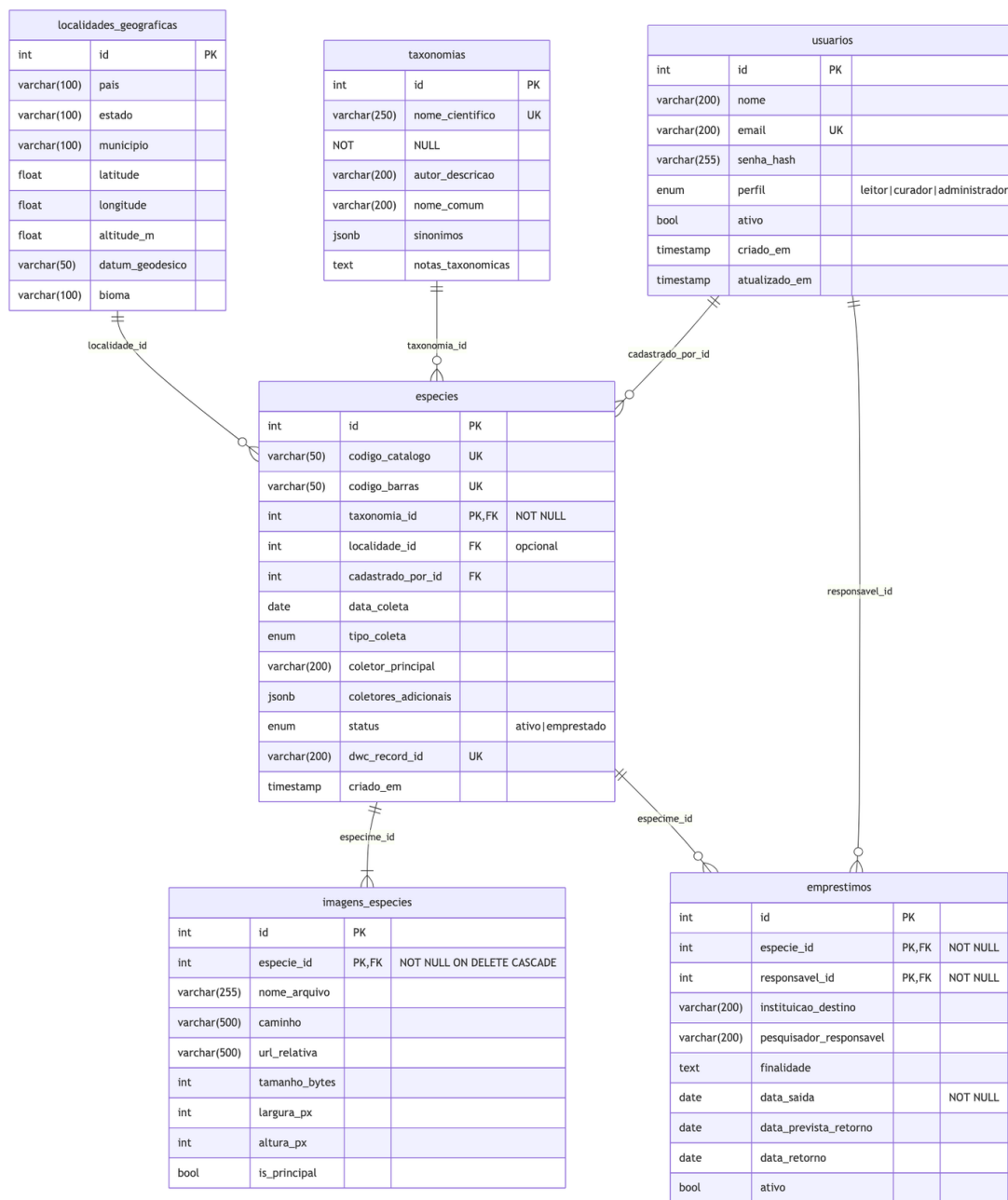


Figure 4: Diagrama ER do PostgreSQL

Projeto Arquitetural

1. Visão geral

O BioAcervo é uma API RESTful em FastAPI com persistência PostgreSQL (SQLAlchemy ORM assíncrono). Ver [Architecture/00_Arquitetura.md](#) para detalhe em camadas C4.

2. Stack

- **API:** FastAPI + Pydantic v2 (DTOs)

- **ORM:** SQLAlchemy assíncrono (asyncpg)
- **Banco:** PostgreSQL 16 (container bioacervo-database)
- **Auth:** JWT Bearer Token (bcrypt + `get_current_user`, `require_roles`)
- **Armazenamento:** bucket local (volume Docker) para imagens
- **Orquestração:** Docker Compose (database + backend + bucket + frontend)

3. Camadas (C4 — Nível de Contêiner)

Contêiner	Tecnologia	Responsabilidade
Frontend	Nginx + React	UI de consulta/curadoria
Backend (API)	FastAPI + Uvicorn	Rotas, auth, orquestração de domínio
Banco	PostgreSQL 16	Persistência relacional
Bucket	Volume local	Armazenamento de imagens

4. Camadas internas (Backend)

- `api/v1/endpoints/` — adaptadores HTTP (auth, especimes, taxonomia_localidade, usuarios_emprestimos)
- `services/` — regras de negócio (EspecimeService, UsuarioService, ImagemService, ExportService + EtiquetaService)
- `models/` — ORM (6 entidades + 3 enums)
- `schemas/` — DTOs Pydantic v2
- `core/` — security (JWT), config (settings)
- `db/session.py` — engine assíncrono + `get_db`

5. Fluxo de requisição

1. Endpoint recebe requisição, autentica (JWT) e valida entrada (Pydantic)
2. Chama Service correspondente
3. Service orquestra regras de negócio e acesso a dados (ORM)
4. Retorna DTO serializado

7. Decisões arquiteturais

- ORM assíncrono para concorrência em I/O de banco
- Controle de acesso no backend (`require_roles`), não na UI
- Exportação DwC-A desacoplada em `ExportService` (interoperabilidade com GBIF/iDigBio/SpeciesLink)
- Bucket local como volume, facilitando migração futura para objeto store

5. Diagramas

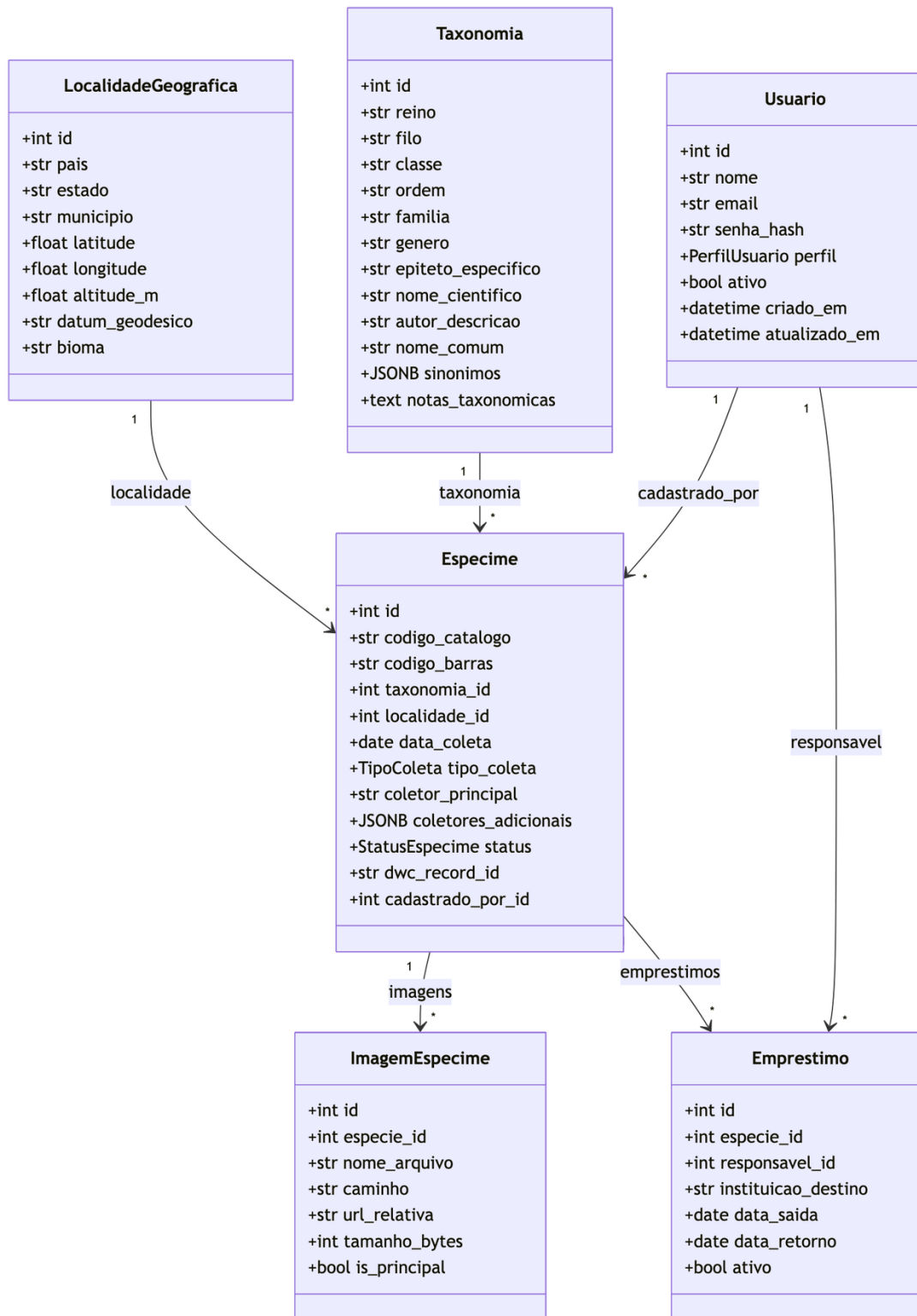


Figure 5: Diagrama de Classes

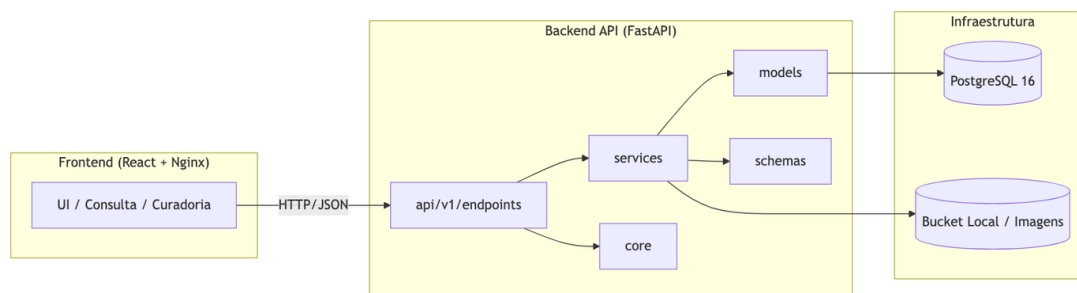


Figure 6: Diagrama de Pacotes/Componentes

Padrões de Projeto e Diretrizes de Extensão

1. Objetivo

Consolidar padrões de implementação observados no backend do BioAcervo, para guiar evolução futura sem regressão.

2. Padrões de backend

- **DTOs Pydantic v2** em `schemas/` separam validação de persistência (`models.py`)
- **Services estáticos** (`@staticmethod`) em `services/` concentram regras de negócio
- **Endpoints magros**: autenticam, validam e delegam a Service; não contêm regra de negócio
- **Async ao longo de toda a pilha**: `async def` + `AsyncSession` + `asyncpg`
- `get_db` como dependency do FastAPI injeta a sessão (`commit/rollback/close`)

3. Modelo de dados

- Herança de `Base` (SQLAlchemy Declarative)
- **Cascade** em relações críticas (`ImagemEspecime` → `Especime` ON DELETE CASCADE)
- Campos JSONB para dados flexíveis (`sinonimos`, `coletores_adicionais`, `referencias`)
- `UniqueConstraints` em identificadores (`email`, `nome_cientifico`, `codigo_catalogo`, `codigo_barras`, `dwc_record_id`)

4. Autenticação e autorização

- JWT emitido em login; `get_current_user` decodifica o token
- `require_roles(*perfis)` protege endpoints por `PerfilUsuario`
- A decisão de acesso é sempre no backend

5. Exportação

- `ExportService` isola a lógica DwC-A (`occurrence.csv` + `meta.xml` + `eml.xml`)
- Etiquetas via `EtiquetaService` (PDF Code128 a partir de `codigo_barras`)

6. Diretrizes de extensão

- Novos domínios devem seguir a triade: `endpoint` (HTTP) → `service` (regra) → `model` (ORM)
- Validação de entrada sempre em `schemas/` (Pydantic)
- Dados geográficos seguem WGS84 por padrão; bounding box via filtros lat/lon
- Imagens limitadas a 20 MB e armazenadas em bucket local
- Qualquer evolução de modelo deve atualizar `10-analise-e-modelo-conceitual.md` e `08-regras-de-negocio.md`

Rastreabilidade e Priorização

1. Objetivo

Consolidar a matriz de rastreabilidade entre Requisitos Funcionais (RF), Casos de Uso (UC) e Regras de Negócio (RN), extraída do código-fonte.

2. Matriz RF → UC → RN

Requisito	Caso de Uso	Regra de Negócio	Evidência (arquivo:linha)
RF-01 Cadastro de usuário	UC-01	RN-02	usuarios_emprestimos.py:39
RF-03 Autenticação JWT	UC-02	RN-01	core/security.py:get_current_user
RF-05 Criação só por admin	UC-01	RN-02	usuarios_emprestimos.py:43
RF-08 CRUD Taxonomia	UC-04	RN-06, RN-07	taxonomia_localidade.py:53,57
RF-10 CRUD Localidade	UC-05	RN-07, RN-08	taxonomia_localidade.py:129,133
RF-12 Cadastro Espécime (códigos auto)	UC-06	RN-09, RN-11	especime_service.py:46-54
RF-13 Taxonomia FK NOT NULL	UC-06	RN-09	models.py:Especime
RF-15 Bounding box	UC-07	RN-10	especime_service.py:125-133
RF-19 Upload imagem	UC-08	RN-13	imagem_service.py
RF-21 Criar Empréstimo	UC-09	RN-14, RN-15	usuarios_emprestimos.py:125,134,141
RF-24 Retornar Empréstimo	UC-10	RN-16	usuarios_emprestimos.py:162-166
RF-25 Export DwC-A	UC-11	RN-18	export_service.py

3. Priorização inicial

Prioridade	Itens	Motivo
Alta	RF-12, RF-21, RF-24, RN-09/14/15/16	Núcleo de curadoria e disponibilidade
Média	RF-08/10, RF-15, RN-06/07	Padronização taxonômica/geográfica
Média	RF-25, RN-18	Interoperabilidade com portais
Baixa	RF-19/20	Apoio visual (imagens/etiquetas)

4. Observações

- `models.py` é a fonte de verdade para atributos e restrições (UNIQUE, FK, JSONB)
 - `endpoints/` é a fonte para casos de uso e regras de acesso (`require_roles`)
 - `services/` concentra as regras de negócio (RN-11 geração de códigos, RN-14/15/16 ciclo de empréstimo)
-